

Devoir 2

Algorithmique sur les Graphes et Combinatoire

Master 1 - I2A

Université de Franche-Comté

N'jie ZAMON

Année universitaire 2025-2026

6 janvier 2026



Table des matières

1	Exercice 1 : Balles rebondissantes	2
1.1	Question 1.1 : Stratégie avec une balle	2
1.2	Question 1.2 : Nombre minimum d'essais avec une balle	2
1.3	Question 1.3 : Stratégie avec deux balles pour $P = 10$	2
1.4	Question 1.4 : Relation de récurrence pour P	4
1.5	Question 1.5 : Algorithme de programmation dynamique	4
1.6	Question 1.6 : Stratégie générale pour trouver $p_{\max}$	5
2	Exercice 2 : Plus petit cycle dans un graphe	6
2.1	Question 2.1 : Solution pour trouver le plus petit cycle	6
2.2	Question 2.2 : Algorithme <code>ppCycle(G[])</code>	6
2.3	Question 2.3 : Cet algorithme est-il optimal ?	7
3	Exercice 3 : Liste de commandes	8
3.1	Question 3.1 : Ordonnancement minimisant les pénalités	8
3.2	Question 3.2 : Solution gloutonne	8
3.3	Question 3.3 : Algorithme qui implémente la solution	9
3.4	Question 3.4 : Votre solution est-elle optimale ?	9

1 EXERCICE 1 : BALLES REBONDISSANTES

1 Exercice 1 : Balles rebondissantes

1.1 Question 1.1 : Stratégie avec une balle

Stratégie proposée

Avec une seule balle, la seule stratégie viable pour être certain de trouver p_{max} est de procéder à des essais séquentiels en partant de la plus petite distance. On lance la balle à la distance $p = 1$, puis $p = 2$, puis $p = 3$, et ainsi de suite, jusqu'à ce que la balle ne soit plus rattrapée. La valeur de p_{max} est alors la dernière distance à laquelle la balle a été rattrapée.

Justification : Cette approche est une **recherche linéaire**. Elle est la seule garantie avec une seule balle car si nous effectuions un saut (par exemple, on teste $p = 5$ directement et la balle est perdue), nous n'aurions aucun moyen de savoir si p_{max} était 1, 2, 3 ou 4. Nous aurions perdu notre seule balle sans pouvoir déterminer la valeur exacte de p_{max} . La stratégie séquentielle explore chaque possibilité de manière exhaustive et ordonnée, ce qui assure de trouver la valeur exacte de p_{max} sans risque de perdre l'unique balle avant d'avoir testé toutes les distances inférieures (voir Figure 1).

Schéma de la situation physique

La Figure 1 illustre les éléments clés du problème : la personne qui lance, le mur, la distance p qui les sépare, et la trajectoire de la balle.

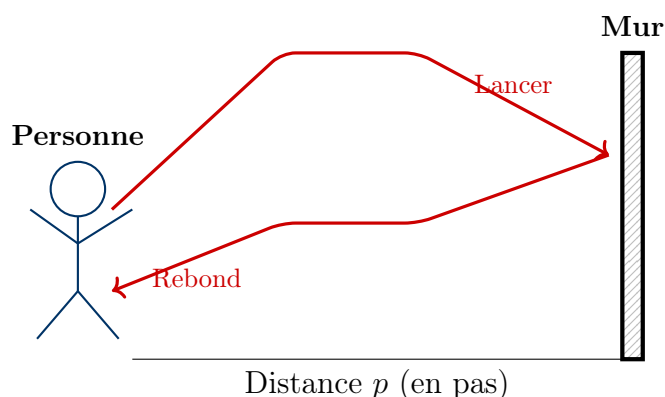


FIGURE 1 – Illustration de la situation physique du lancer de balle

1.2 Question 1.2 : Nombre minimum d'essais avec une balle

Le nombre minimum d'essais nécessaires dans le pire des cas est P .

Justification : Le pire des cas se produit lorsque $p_{max} = P - 1$. Dans ce scénario, nous devons tester avec succès toutes les distances de 1 à $P - 1$. Cela nécessite $P - 1$ essais. Un dernier essai à la distance P confirmera que la balle n'est plus rattrapée. Au total, cela fait P essais. Il n'est pas possible de faire mieux, car chaque essai avec une seule balle ne peut explorer qu'une seule nouvelle distance et nous ne pouvons pas sauter de distances sous peine de perdre la balle sans obtenir l'information souhaitée.

1.3 Question 1.3 : Stratégie avec deux balles pour $P = 10$

L'idée est d'utiliser la première balle pour réduire l'intervalle de recherche de manière efficace, et la deuxième balle pour explorer cet intervalle de manière linéaire.

1 EXERCICE 1 : BALLES REBONDISSANTES

Stratégie et Justification

L'objectif est de minimiser le nombre d'essais **dans le pire des cas**. Appelons ce nombre e . Si la première balle est perdue au k -ième essai, il nous restera $e - k$ essais pour explorer l'intervalle en dessous avec la deuxième balle. Pour que le nombre total d'essais soit constant (égal à e) quel que soit l'endroit où la première balle est perdue, la taille de l'intervalle exploré à chaque étape doit diminuer de 1.

La stratégie optimale est donc de faire des sauts de taille e , puis $e - 1$, puis $e - 2$, etc. Le nombre total de pas P que l'on peut couvrir avec e essais est la somme :

$$P = e + (e - 1) + (e - 2) + \dots + 1 = \frac{e(e + 1)}{2}$$

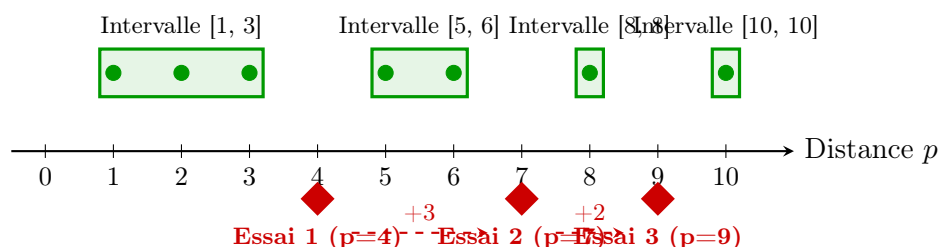
Nous cherchons le plus petit e tel que cette somme soit supérieure ou égale à 10.

- Si $e = 3$, $\frac{3 \times 4}{2} = 6$ (insuffisant pour couvrir $P = 10$).
- Si $e = 4$, $\frac{4 \times 5}{2} = 10$ (suffisant).

Le nombre minimal d'essais dans le pire des cas est donc $e = 4$. Les points de lancement pour la première balle sont déterminés en partant de 0 et en ajoutant les pas décroissants :

- Pas 1 : $0 + 4 = 4$
- Pas 2 : $4 + 3 = 7$
- Pas 3 : $7 + 2 = 9$
- Pas 4 : $9 + 1 = 10$ (cas où la balle n'est jamais perdue).

La stratégie est donc de lancer la première balle aux distances $p = 4, p = 7$, puis $p = 9$.



- ◆ Balle 1 : Recherche par sauts (coûte 1 essai)
- Balle 2 : Recherche linéaire (coûte 1 essai par pas)

FIGURE 2 – Stratégie à deux balles pour $P=10$: visualisation des points de test et des intervalles de recherche

Déroulement de la stratégie (pire cas = 4 essais) :

1. **Essai 1 (Balle 1 à $p = 4$) :**
 - **Si perdue :** $p_{max} \in \{0, 1, 2, 3\}$. On utilise la balle 2 pour tester $p = 1, 2, 3$. Total : $1 + 3 = 4$ essais.
 - **Si rattrapée :** On continue.
2. **Essai 2 (Balle 1 à $p = 7$) :**
 - **Si perdue :** $p_{max} \in \{4, 5, 6\}$. On utilise la balle 2 pour tester $p = 5, 6$. Total : $2 + 2 = 4$ essais.
 - **Si rattrapée :** On continue.
3. **Essai 3 (Balle 1 à $p = 9$) :**

1 EXERCICE 1 : BALLES REBONDISSANTES

- **Si perdue** : $p_{max} \in \{7, 8\}$. On utilise la balle 2 pour tester $p = 8$. Total : $3 + 1 = 4$ essais.
- **Si rattrapée** : $p_{max} \in \{9, 10\}$. On utilise la balle 2 pour tester $p = 10$. Total : $3 + 1 = 4$ essais.

Conclusion : Quelle que soit la valeur de p_{max} , le nombre d'essais ne dépassera pas 4. C'est donc le nombre minimal d'essais dans le pire des cas.

1.4 Question 1.4 : Relation de récurrence pour P

Soit $P(n, e)$ la taille maximale de la pièce que l'on peut explorer avec n balles et e essais. On effectue le premier essai à une distance d .

- **Si la balle est perdue** : Il nous reste $n-1$ balles et $e-1$ essais pour explorer les $d-1$ étages inférieurs. La taille maximale est $P(n-1, e-1)$. On doit donc avoir $d-1 = P(n-1, e-1)$, soit $d = P(n-1, e-1) + 1$.
- **Si la balle est rattrapée** : Il nous reste n balles et $e-1$ essais pour explorer les étages supérieurs. La taille maximale est $P(n, e-1)$.

La taille maximale $P(n, e)$ est la somme de la taille explorée en dessous de d , de l'étage d lui-même, et de la taille explorable au-dessus de d .

La relation de récurrence est donc :

$$P(n, e) = P(n-1, e-1) + 1 + P(n, e-1)$$

Avec les conditions initiales :

- $P(1, e) = e$ (avec une balle, on peut tester e étages linéairement).
- $P(n, 1) = 1$ (avec un seul essai, on ne peut tester qu'un seul étage).

1.5 Question 1.5 : Algorithme de programmation dynamique

Cet algorithme implémente la relation de récurrence de manière itérative (bottom-up), comme présenté dans le cours (Chapitre 3 : Programmation dynamique).

```

1 Fonction tailleMaxPiece(n, e) : Entier
2   // Cr er un tableau P de taille (n+1) x (e+1)
3   P = tableau[(n+1)][(e+1)]
4
5   // Initialiser les conditions aux limites (cas de base)
6   // P(1, e) = e
7   Pour j de 1 à e:
8     P[1][j] = j
9
10  // P(n, 1) = 1
11  Pour i de 1 à n:
12    P[i][1] = 1
13
14  // Remplir le tableau en utilisant la relation de r currence
15  Pour i de 2 à n: // Pour chaque nombre de balles
16    Pour j de 2 à e: // Pour chaque nombre d'essais
17      P[i][j] = P[i-1][j-1] + 1 + P[i][j-1]
18
19  Retourner P[n][e]
20 Fin Fonction

```

Listing 1 – Fonction `tailleMaxPiece` par programmation dynamique

1.6 Question 1.6 : Stratégie générale pour trouver p_{max}

La stratégie pour trouver p_{max} est une construction directe basée sur la logique de la programmation dynamique.

1. **Déterminer le nombre d'essais e** : Utiliser l'algorithme de la question 1.5 pour trouver le plus petit e tel que $P(n, e) \geq P$.
2. **Effectuer les essais** :
 - Le premier essai avec la première balle se fait à la distance $d_1 = P(n - 1, e - 1) + 1$.
 - Si la balle est rattrapée, le problème se réduit à trouver p_{max} dans une pièce de taille $P - d_1$ avec n balles et $e - 1$ essais. Le prochain essai se fera à $d_2 = d_1 + P(n - 1, e - 2) + 1$.
 - Si la balle est perdue au premier essai, le problème se réduit à trouver p_{max} dans une pièce de taille $d_1 - 1$ avec $n - 1$ balles et $e - 1$ essais. On utilise alors la deuxième balle pour une recherche séquentielle à partir de $p = 1$.

Cette stratégie garantit de trouver p_{max} en au plus e essais.

2 EXERCICE 2 : PLUS PETIT CYCLE DANS UN GRAPHE

2 Exercice 2 : Plus petit cycle dans un graphe

2.1 Question 2.1 : Solution pour trouver le plus petit cycle

Solution proposée

Pour trouver le plus petit cycle (la "circonférence" ou *girth* en anglais) dans un graphe non pondéré, on peut utiliser une variante de l'algorithme de parcours en largeur (BFS - Breadth-First Search, voir cours 1.3.5). L'idée est de lancer un BFS à partir de chaque sommet et de chercher le plus court cycle qui inclut ce sommet.

Principe : Pour un sommet de départ s :

1. Lancer un BFS depuis s . Le BFS explore les sommets par couches de distance croissante de s .
2. Pendant le parcours, si on rencontre une arête (u, v) qui relie deux sommets u et v déjà visités et qui ne sont pas dans une relation parent-enfant dans l'arbre de BFS, alors on a trouvé un cycle.
3. La longueur de ce cycle est $\text{distance}(s, u) + \text{distance}(s, v) + 1$.

On répète ce processus pour chaque sommet du graphe et on garde le cycle le plus court trouvé.

Illustration sur le graphe de la figure 1 :

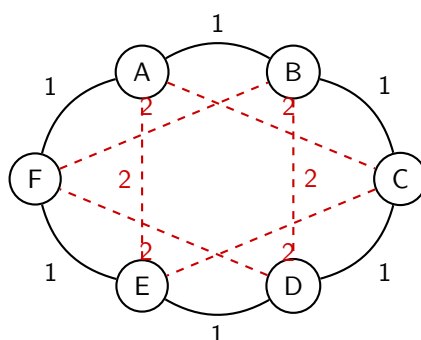


FIGURE 3 – Graphe d'exemple pour la recherche du plus petit cycle

En partant de A, on trouve B (dist 1), puis C (dist 2), puis D (dist 3), etc. Si une arête relie deux sommets de la même "couche" ou de couches adjacentes, on a trouvé un cycle. Par exemple, si une arête relie A et C, on a un cycle A-B-C-A de longueur 3.

2.2 Question 2.2 : Algorithme `ppCycle(G[])`

Voici un algorithme qui implémente la stratégie décrite ci-dessus. Il utilise les structures de listes et files mentionnées dans le sujet et le cours.

```

1 Fonction ppCycle(G[]) : Entier
2   // G est un tableau de listes de voisins
3   n = longueur(G) // Nombre de sommets
4   longueurMin = infinie
5
6   // Pour chaque sommet du graphe, on le considère comme racine d'un BFS
7   Pour i de 0 à n-1:
8     // Initialiser les tableaux de distance et de parent pour ce BFS
9     distance = tableau[n]
10    parent = tableau[n]
11    Pour j de 0 à n-1:
12      distance[j] = -1

```

2 EXERCICE 2 : PLUS PETIT CYCLE DANS UN GRAPHE

```
13     parent[j] = -1
14
15     // Lancer le BFS      partir du sommet i
16     file = nouvelleFile()
17     enfiler(file, i)
18     distance[i] = 0
19
20     Tant que file n'est pas vide:
21         u = defiler(file)
22
23         // Pour chaque voisin v de u
24         Pour chaque v dans G[u]:
25             Si distance[v] == -1:
26                 // Si v n'a pas t visité, on le découvre
27                 distance[v] = distance[u] + 1
28                 parent[v] = u
29                 enfiler(file, v)
30             Sinon si parent[u] != v:
31                 // Si v a t visité et n'est pas le parent de u,
32                 // on a trouvé un cycle.
33                 longueurCycle = distance[u] + distance[v] + 1
34
35                 Si longueurCycle < longueurMin:
36                     longueurMin = longueurCycle
37
38     Retourner longueurMin
39 Fin Fonction
```

Listing 2 – Fonction ppCycle pour trouver la longueur du plus petit cycle

2.3 Question 2.3 : Cet algorithme est-il optimal ?

Analyse de la complexité :

- La boucle externe s'exécute n fois (pour chaque sommet).
- À l'intérieur, on effectue un parcours en largeur (BFS). La complexité d'un BFS est $O(V + E)$, où V est le nombre de sommets et E le nombre d'arêtes.
- La complexité totale de l'algorithme est donc $O(V \times (V + E))$.

Optimalité : Cet algorithme est efficace et constitue une approche standard pour ce problème. Pour les graphes denses (où E est proche de V^2), la complexité est $O(V^3)$. Cela est comparable à l'algorithme de Floyd-Warshall (voir cours 1.5.2), qui a une complexité de $O(V^3)$, pour trouver les plus courts chemins entre toutes les paires et peut être adapté pour trouver le plus court cycle.

Pour les graphes peu denses (où E est proche de V), la complexité est $O(V^2)$.

Peut-on faire mieux asymptotiquement ? Il n'existe pas d'algorithme connu qui résolve le problème en sous-temps quadratique pour les graphes généraux. La complexité de $O(V \times (V + E))$ est donc considérée comme **efficace et standard** pour ce problème. On peut donc le considérer comme optimal dans le sens où il utilise une méthode de parcours (BFS) qui est optimale pour la recherche de plus courts chemins à partir d'une source, et il l'applique de manière exhaustive pour garantir de trouver le plus petit cycle.

3 EXERCICE 3 : LISTE DE COMMANDES

3 Exercice 3 : Liste de commandes

3.1 Question 3.1 : Ordonnancement minimisant les pénalités

Ce problème est un exemple classique d'ordonnancement avec pénalités, qui peut être résolu par un **algorithme glouton** (voir cours Chapitre 4 : Programmation gloutonne).

Stratégie gloutonne

1. Trier les commandes par date limite d_i croissantes.
2. Itérer sur les commandes triées.
3. Pour chaque commande i , tenter de la planifier.
 - Si sa date limite d_i n'est pas encore dépassée (c'est-à-dire si le nombre de commandes déjà planifiées est $< d_i$), on l'ajoute à l'emploi du temps.
 - Si sa date limite est dépassée, on ne peut pas la prendre. On regarde la pénalité p_i de cette commande. Si p_i est supérieure à la pénalité la plus faible parmi les commandes déjà planifiées, on remplace la commande à plus faible pénalité par la commande i .

Application au tableau :

1. **Trier par d_i croissant :** $c1(d=2, p=4)$, $c2(d=2, p=3)$, $c3(d=2, p=1)$, $c4(d=3, p=3)$, $c5(d=4, p=2)$, $c6(d=5, p=3)$, $c7(d=6, p=2)$, $c8(d=6, p=1)$.
2. **Ordonnancement :**
 - **t=1 :** On prend $c1$. Emploi du temps : $[c1]$.
 - **t=2 :** On prend $c2$. Emploi du temps : $[c1, c2]$.
 - **t=3 :** On considère $c3$ ($d=2, p=1$). Sa date limite (2) est dépassée. Sa pénalité (1) est inférieure à toutes les pénalités dans l'emploi du temps (3 et 4). On ne la remplace pas.
 - **t=4 :** On prend $c4$. Emploi du temps : $[c1, c2, c4]$.
 - **t=5 :** On prend $c5$. Emploi du temps : $[c1, c2, c4, c5]$.
 - **t=6 :** On prend $c6$. Emploi du temps : $[c1, c2, c4, c5, c6]$.
 - **t=7 :** On prend $c7$. Emploi du temps : $[c1, c2, c4, c5, c6, c7]$.
 - **t=8 :** On considère $c8$ ($d=6, p=1$). Sa date limite (6) est dépassée. Sa pénalité (1) est inférieure à la pénalité la plus faible dans l'emploi du temps (2). On ne la remplace pas.

Ordonnancement final : Les commandes réalisées sont $c1, c2, c4, c5, c6, c7$.

Commandes non réalisées : $c3$ et $c8$.

Pénalité totale minimale : $p_3 + p_8 = 1 + 1 = 2$.

3.2 Question 3.2 : Solution gloutonne

La solution gloutonne est celle décrite dans la question précédente.

1. Trier les commandes C par date limite d_i croissantes.
2. Initialiser une liste vide CR pour les commandes réalisées.
3. Parcourir les commandes triées une par une. Pour chaque commande i :
 - a. Si le nombre de commandes dans CR est $< d_i$, ajouter i à CR .
 - b. Sinon, trouver la commande j dans CR qui a la plus petite pénalité p_j .
 - c. Si la pénalité p_i de la commande i est strictement supérieure à p_j , remplacer la commande j par la commande i dans CR .

Le critère glouton est de toujours privilégier les commandes avec les dates limites les plus strictes, et en cas de conflit, de garder celles qui ont les pénalités les plus élevées.

3 EXERCICE 3 : LISTE DE COMMANDES

3.3 Question 3.3 : Algorithme qui implémente la solution

Voici l'algorithme en pseudo-code, utilisant les fonctions fournies et les structures de listes.

```

1 Fonction minimiserPenalites(C[]) : Liste
2   // C est un tableau de n commandes, avec C[i].d et C[i].p
3
4   // 1. Trier les commandes par date limite croissante
5   indicesTries = comParD(C, [0, 1, ..., longueur(C)-1])
6
7   // 2. Initialiser la liste des commandes réalisées CR
8   // On utilise une liste de paires (indice, penalite)
9   CR = nouvelleListe()
10
11  // 3. Parcourir les commandes triées
12  Pour chaque indice i dans indicesTries:
13    // 3a. Si on peut ajouter la commande
14    Si longueur(CR) < C[i].d:
15      ajouterFin(CR, (i, C[i].p))
16    // 3b. Sinon, vérifier si on peut remplacer une commande
17    Sinon:
18      // Trouver la commande avec la pénalité minimale dans CR
19      minPenalite = infinie
20      indexMinDansCR = -1
21      Pour k de 0 à longueur(CR)-1:
22        commandeCourante = obtenirElement(CR, k)
23        Si commandeCourante.p < minPenalite:
24          minPenalite = commandeCourante.p
25          indexMinDansCR = k
26
27      // 3c. Remplacer si la pénalité est plus intéressante
28      Si C[i].p < minPenalite:
29        remplacerElement(CR, indexMinDansCR, (i, C[i].p))
30
31  // Retourner la liste des indices des commandes réalisées
32  listeFinale = nouvelleListe()
33  Pour chaque element dans CR:
34    ajouterFin(listeFinale, element.indice)
35
36  Retourner listeFinale
37 Fin Fonction

```

Listing 3 – Algorithme glouton pour minimiser les pénalités

3.4 Question 3.4 : Votre solution est-elle optimale ?

Oui, la solution gloutonne est optimale.

Preuve (par argument d'échange) : La preuve s'appuie sur le concept de **propriété du choix glouton** et de **sous-structure optimale** (voir cours 4.5).

1. **Sous-structure optimale :** Si nous avons un ordonnancement optimal S_{opt} pour un sous-ensemble de commandes, l'ordonnancement de ce sous-ensemble doit être optimal pour ce sous-ensemble.
2. **Propriété du choix glouton :** Montrons qu'il existe toujours une solution optimale qui contient le premier choix fait par l'algorithme glouton.

Soit S_g l'ensemble des commandes retournées par l'algorithme glouton, et S_{opt} un ensemble optimal. Supposons que les commandes sont triées par date limite $d_1 \leq d_2 \leq \dots \leq d_n$.

Considérons la première commande k pour laquelle l'algorithme glouton et une solution optimale diffèrent.

3 EXERCICE 3 : LISTE DE COMMANDES

- L'algorithme glouton a sélectionné un ensemble de $k - 1$ commandes, noté G_{k-1} , qui ont les dates limites les plus basses.
- La solution optimale S_{opt} contient aussi $k - 1$ commandes parmi les $k - 1$ premières.
- L'algorithme glouton considère la k -ième commande, c_k .
 - **Cas 1** : L'algorithme ajoute c_k à G_{k-1} . Cela signifie qu'il y a une place libre. Toute solution optimale peut aussi ajouter c_k sans violer de contrainte. Donc il existe une solution optimale contenant $G_{k-1} \cup \{c_k\}$.
 - **Cas 2** : L'algorithme remplace une commande c_j (avec la pénalité la plus faible) par c_k . Cela signifie que $p_k > p_j$. Soit S_{opt} une solution optimale. Si S_{opt} contient déjà c_k , alors G_{k-1} est un sous-ensemble de S_{opt} . Sinon, S_{opt} doit contenir k commandes choisies parmi les k premières. L'une d'elles est c_j (ou une commande avec une pénalité encore plus faible). En remplaçant cette commande dans S_{opt} par c_k , nous obtenons un nouvel ensemble S' dont la pénalité totale n'est pas pire que celle de S_{opt} (car $p_k > p_j$). S' est donc aussi une solution optimale et elle contient le choix glouton.

Par itération, on peut transformer n'importe quelle solution optimale en la solution gloutonne sans dégrader la pénalité totale. Cela prouve que la solution gloutonne est optimale.